

Rockchip Developer Guide Android ARC

文件标识：RK-KF-YF-E08

发布版本：V1.0.0

日期：2024-08-19

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自所有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文主要介绍 Android ARC 模块，开发或技术支持人员通过阅读此文档，对 Android 平台 ARC 模块有一个初步的了解，帮助读者开发和调试。

产品版本

芯片名称	内核版本
适配所有芯片	适配所有版本

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	张贵朋	2024-08-19	初始版本

目录

Rockchip Developer Guide Android ARC

简介

- 引脚定义
- CEC 相关的 ARC 命令

适配

- 配置 CEC 设备类型
- 配置 CEC 端口信息
- 配置声卡
- 配置 Android Audio
 - Audio Policy 声明 ARC
 - Audio Policy 配置路由
 - Audio HAL 绑定声卡

验证

- 确认 ARC 链路状态
- 确认声卡可用性
- 确认 Audio Policy 可用设备
- 确认 Audio Policy 路由状态

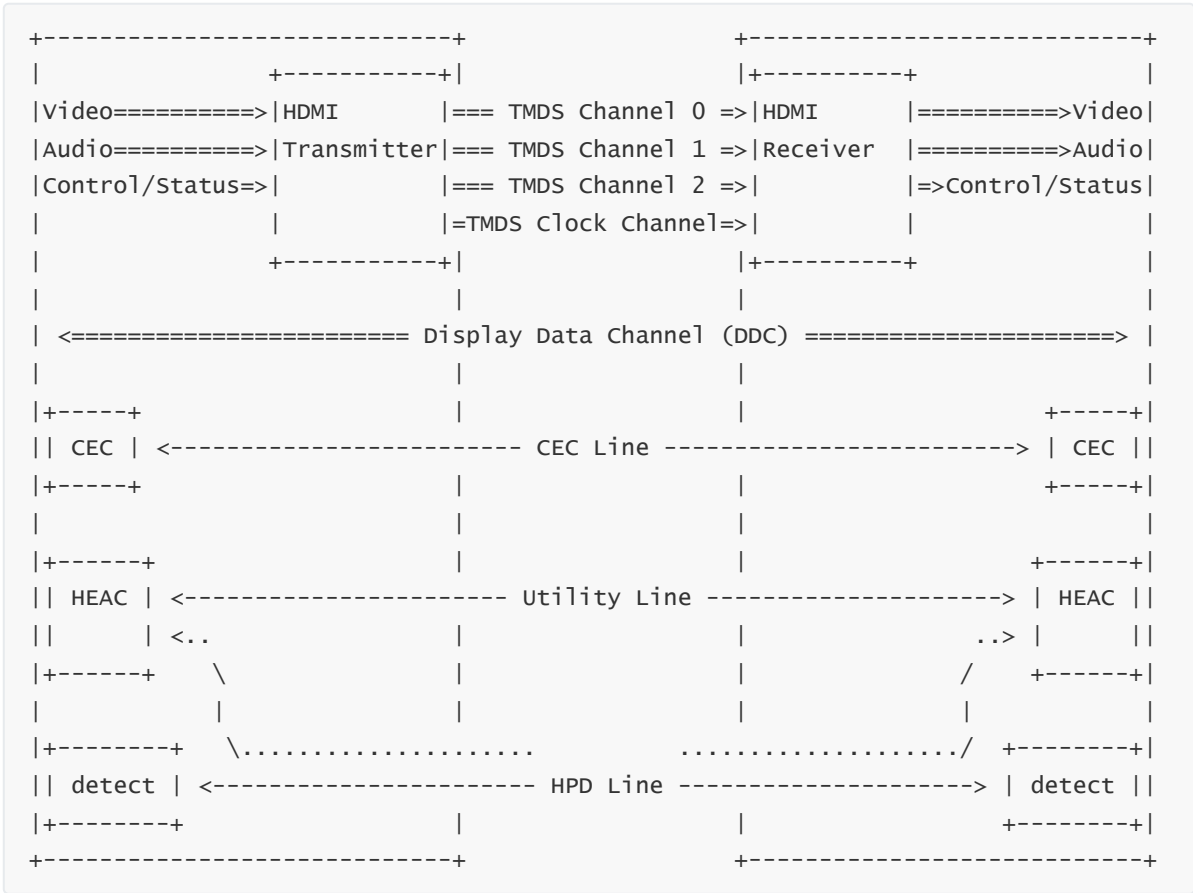
Q&A

- CEC 相关问题
 - CEC 没有启用
 - 没有 CEC 消息的收发
 - 没有收到功放或回音壁音响的 CEC 消息
 - 收到 <Initiate ARC> 后回复 <Feature Abort>
 - 功放或回音壁没有发起 <Initiate ARC>
 - HPD 频繁被 toggle, 声音无法保持输出
- Audio 相关问题
 - 无法生成 ARC 的声卡
 - 通过 tinyply 指定 ARC 的声卡播放音频, 功放或回音壁音响没有输出

简介

ARC (Audio Return Channel) 属于 HEAC (HDMI Ethernet and Audio Return Channel) feature 的一部分, 在 1.4b 中定义

在原有的 HDMI 端口中的一个预留脚 (Utility Line) 上回传 S-PDIF 信号, 因此 ARC 的传输方向与 TMDS 方向相反



引脚定义

参考 [HDMI 1.4b Specification](#) HEAC 2.4 小节

Transmission Signal	Required Condition
HEC	HEAC+ = High , HEAC- = High, +5V Power = High
ARC (Common mode)	HEAC+ = High , HEAC- = High, +5V Power = High
ARC (Single mode)	HEAC+ = ARC (Single mode), HEAC- = normal HPD Signal, +5V Power = High

CEC 相关的 ARC 命令

ARC 的建立流程在 HPD 被拉高后, 通过 CEC 发起, 相关命令如下:

初始化命令:

- <Initiate ARC>

- <Report ARC Initiated>
- <Request ARC Initiation>

终止命令:

- <Terminate ARC>
- <Report ARC Terminated>
- <Request ARC Termination>

命令描述:

Opcode	value	Description	Parameters	Response	Addressing		Mandatory for	
					Direct	Broadcast	Initiator	Follower
<Initiate ARC>	0xC0	Used by an ARC Rx device to activate the ARC functionality in an ARC Tx device.	None	The ARC functionality in the ARC Tx device is activated	Y		ARC Rx device	ARC Tx device
<Report ARC Initiated>	0xC1	Used by an ARC Tx device to indicate that its ARC functionality has been activated.	None		Y		ARC Tx device	ARC Rx device
<Report ARC Terminated>	0xC2	Used by an ARC Tx device to indicate that its ARC functionality has been deactivated.	None		Y		ARC Tx device	ARC Rx device
<Request ARC Initiation>	0xC3	Used by an ARC Tx device to request an ARC Rx device to activate the ARC functionality in the ARC Tx device.	None	ARC Rx device sends an <Initiate ARC> message	Y			ARC Rx device
<Request ARC Termination>	0xC4	Used by an ARC Tx device to request an ARC Rx device to deactivate the ARC functionality in the ARC Tx device.	None	ARC Rx device sends an <Terminate ARC> message	Y			ARC Rx device

Opcode	value	Description	Parameters	Response	Addressing		Mandatory for	
<Terminate ARC>	0xC5	Used by an ARC Rx device to deactivate the ARC functionality in an ARC Tx device.	None	The ARC functionality in the ARC Tx device is deactivated.	Y		ARC Rx device	ARC Tx device

适配

本文档均围绕 ARC TX 讨论, 即调试设备作为 TV, 而 ARC RX 同理

ARC 依赖 CEC 建议物理链路, 因此除了 Audio 相关的配置以外, 还需要 CEC 相关的配置

配置 CEC 设备类型

修改设备在 CEC 中的充当的设备类型, 作为 TV 设备使用

属性的值代表的设备类型如下:

```
enum class CecDeviceType : int32_t {
    INACTIVE = -1 /* -1 */,
    TV = 0, /* 电视 */
    RECORDER = 1,
    TUNER = 3,
    PLAYBACK = 4, /* 播放盒子 */
    AUDIO_SYSTEM = 5, /* 功放或 soundbar */
    MAX = 5 /* AUDIO_SYSTEM */,
};
```

1. Android 14 以下

device/rockchip/common/device.mk

```
#注释掉下面的条件限制
#ifneq ($(filter atv box, $(strip $(TARGET_BOARD_PLATFORM_PRODUCT))), )
#...
#修改设备类型
PRODUCT_PROPERTY_OVERRIDES += ro.hdmi.device_type=0
#...
#注释掉下面的条件限制
#endif
```

2. Android 14 及以上

在对应项目使用的 BoardConfig.mk 加入下面的编译选项

```
BOARD_SUPPORT_HDMI_CEC := true
BOARD_HDMI_CEC_TYPE := 0
BOARD_HDMI_IN_SUPPORT := true
```

可以通过使用的芯片方案和 lunch 时选择的 product 确认 BoardConfig.mk 路径

比如芯片名称为 rk3588, product 为 rk3588_u

```
# echo $TARGET_PRODUCT
# rk3588_u
```

则 BoardConfig.mk 的位置为 device/rockchip/rk3588/rk3588_u/BoardConfig.mk

配置 CEC 端口信息

ARC port 使用的 physical address 必须为直连的地址, 比如 x.0.0.0

HDMI RX/switch 在分配 physical address 的时候需要注意, 否则会导致某些功放无法进行后续的 ARC 协商

项目中可能遇到的拓扑:

```
SOC HDMI RX (0.0.0.0)
├─ HDMI Switch 1 (3.0.0.0)
│   ├── OPS (3.1.0.0)
│   │
│   ├── HDMI Switch 2 (3.2.0.0)
│   │   ├── USB C1 (3.2.1.0)
│   │   ├── HDMI1 (1.0.0.0) - ARC port
│   │   └── DP (3.2.3.0)
│   │
│   └─ HDMI Switch 3 (3.3.0.0)
│       ├── HDMI3 (3.0.0.0)
│       ├── USB C2 (3.3.2.0)
│       ├── VGA (3.3.3.0)
│       └── HDMI2 (2.0.0.0)
```

各个 port 的属性说明如下:

属性名	说明
type	类型, TMDS 的传输方向, 可以为 HDMI_INPUT 或 HDMI_OUTPUT
port_id	id, 从 1 开始, 比如 1 代表 HDMI "port 1"
cec_supported	是否支持 CEC, 1 为支持; 0 为不支持
arc_supported	是否支持 ARC, 1 为支持; 0 为不支持
physical_address	物理地址, 比如 0x1234 代表 1.2.3.4

在 HDMI CEC HAL 按照如下方法添加或修改 port 信息

1. Android 14 以下参考如下方法添加 ARC port

hardware/rockchip/hdmicec/

```
diff --git a/hdmi_cec.cpp b/hdmi_cec.cpp
index 6f5be34..c955f7d 100644
--- a/hdmi_cec.cpp
+++ b/hdmi_cec.cpp
@@ -394,13 +394,20 @@ static void hdmi_cec_get_port_info(const struct
hdmi_cec_device* dev,
```

```

    } else {
        ALOGE("%s open HDMI_DEV_PATH error", __FUNCTION__);
    }
-   list[0] = &ctx->port;
+   list[0] = &ctx->port[0];
    list[0]->type = HDMI_OUTPUT;
    list[0]->port_id = HDMI_CEC_PORT_ID;
    list[0]->cec_supported = support;
    list[0]->arc_supported = 0;
    list[0]->physical_address = val;
-   *total = 1;
+
+   list[1] = &ctx->port[1];
+   list[1]->type = HDMI_INPUT;
+   list[1]->port_id = 0x000002;
+   list[1]->cec_supported = support;
+   list[1]->arc_supported = 1;
+   list[1]->physical_address = 0x3000;
+   *total = 2;
    }

    static void hdmi_cec_set_option(const struct hdmi_cec_device* dev, int flag, int
value)
diff --git a/hdmicec.h b/hdmicec.h
index 919b91e..e3e5235 100644
--- a/hdmicec.h
+++ b/hdmicec.h
@@ -193,7 +193,7 @@ struct hdmi_cec_context_t {
    /* our private state goes below here */
    event_callback_t event_callback;
    void* cec_arg;
-   struct hdmi_port_info port;
+   struct hdmi_port_info port[2];
    int fd;
    bool enable;
    bool system_control;

```

以上的修改只是作为参考, 需要根据实际硬件配置或项目的需求作调整

2. Android 14 及以上

根据需要, 调整 port 数量

hardware/rockchip/hdmicec/hdmicec.h

```
#define HDMI_PORT_NUM 1
```

根据需要调整 port 属性

hardware/rockchip/hdmicec/hdmi_connection.cpp

```

void hdmi_connection_get_port_info(struct hdmi_connection_context_t* ctx, struct
hdmi_port_info* list[], int* total)
{
    // ...
    for (int i = 0; i < HDMI_PORT_NUM; i++)

```

```

{
#if HDMI_PORT_TYPE == in
    ctx->port[i].type = HDMI_INPUT;
#else
    ctx->port[i].type = HDMI_OUTPUT;
#endif

    ctx->port[i].port_id = i + 1; //start from 1
    ctx->port[i].cec_supported = 1;
    ctx->port[i].arc_supported = 0;
    ctx->port[i].physical_address = 0x1000*(i+1);
}

#if HDMI_PORT_TYPE == in
    // 如果 port 支持 ARC 就更新对应的 arc_supported 为 1
    // 并不一定是默认配置的 port 1, 根据实际需要调整
    // update port which support ARC
    // ctx->port[0].arc_supported = 1;
#endif
    // ...
}

```

配置声卡

声卡配置需要根据实际项目中的硬件设计做调整, 比如硬件接口 simple-audio-card,cpu 和引脚复用 pinctrl-0

参考如下的配置并在项目使用的 dts 中添加

```

/ {
    // ...
    hdmi-arc-sound {
        status = "okay";
        compatible = "simple-audio-card";
        simple-audio-card,name = "rockchip,hdmiaarc";
        simple-audio-card,format = "i2s";
        simple-audio-card,mclk-fs = <128>;
        simple-audio-card,cpu {
            sound-dai = <&spdif_tx1>;
        };
        simple-audio-card,codec {
            sound-dai = <&hdmi_arc_codec>;
        };
    };

    hdmi_arc_codec: hdmi-arc-codec {
        compatible = "rockchip,dummy-codec";
        #sound-dai-cells = <0>;
        sound-name-prefix = "hdmiaarc";
    };
};

&spdif_tx1 {
    pinctrl-names = "default";
    pinctrl-0 = <&spdifm2_tx1>;
    status = "okay";
};

```


修改后会生成 id 为 rockchipdmiarc 的声卡, 通过下面的方法确认

```
# cat /proc/asound/cards
0 [rockchipdmiarc]: simple-card - rockchip,hdmiarc
  rockchip,hdmiarc
...
```

配置 Android Audio

通过 `dumpsys media.audio_policy` 确认当前项目使用的 Audio Policy 配置文件

```
# dumpsys media.audio_policy
// ...
Config source: /vendor/etc/audio_policy_configuration.xml
// ...
```

本小节修改的配置项均围绕当前使用的 Audio Policy 配置文件

Audio Policy 声明 ARC

声明一个名为 HDMI ARC Out 的 devicePort, 作为 Audio HAL (primary) 的输入端

```
<modules>
  <module name="primary" halVersion="2.0">
    <!-- ... -->
    <devicePorts>
      <!-- ... -->
      <devicePort tagName="HDMI ARC Out" type="AUDIO_DEVICE_OUT_HDMI_ARC"
role="sink">
      </devicePort>
    </devicePorts>
    <!-- ... -->
  </module>
</modules>
```

Audio Policy 配置路由

路由 Android Audio Service 的 primary output 到 Audio HAL (primary) 的 HDMI ARC Out

```
<modules>
  <module name="primary" halVersion="2.0">
    <!-- ... -->
    <routes>
      <!-- ... -->
      <route type="mix" sink="HDMI ARC Out"
sources="primary output"/>
    </routes>
    <!-- ... -->
  </module>
</modules>
```

Audio HAL 绑定声卡

绑定 framework 指定的 AUDIO_DEVICE_OUT_HDMI_ARC ("HDMI ARC Out") 到对应的 id 为 rockchipdmiarc 的声卡

hardware/rockchip/audio/tinyalsa_hal/audio_hw.h

```
enum snd_out_sound_cards {  
    // ...  
    // 追加下面这一行  
    SND_OUT_SOUND_CARD_HDMI_ARC,  
    // 保留下面的约束  
    SND_OUT_SOUND_CARD_MAX,  
};
```

hardware/rockchip/audio/tinyalsa_hal/audio_hw.c

```
struct dev_proc_info HDMI_ARC_OUT_NAME[] =  
{  
    {"rockchipdmiarc", NULL,},  
    {NULL, NULL}, /* Note! Must end with NULL, else will cause crash */  
};  
  
static void read_out_sound_card(struct stream_out *out)  
{  
    // ...  
    for (card = 0; card < SNDRV_CARDS; card++) {  
        // ...  
        get_specified_out_dev(&device->dev_out[SND_OUT_SOUND_CARD_HDMI_ARC],  
card, id, HDMI_ARC_OUT_NAME);  
        // ...  
    }  
}  
  
static int start_output_stream(struct stream_out *out)  
{  
    // ...  
    for (int i = 0; i < out->num_configs; ++i) {  
        // ...  
        if (out->devices[i] == AUDIO_DEVICE_OUT_HDMI_ARC) {  
            audio_devices_t route_device = out->devices[i];  
            route_pcm_card_open(adev->dev_out[SND_OUT_SOUND_CARD_HDMI_ARC].card,  
getRouteFromDevice(route_device));  
            card = adev->dev_out[SND_OUT_SOUND_CARD_HDMI_ARC].card;  
            device = adev->dev_out[SND_OUT_SOUND_CARD_HDMI_ARC].device;  
            ret = open_pcm(card, device, (int)SND_OUT_SOUND_CARD_HDMI_ARC, out);  
            if (ret < 0) return ret;  
        }  
        // ...  
    }  
}  
  
static void adev_open_init(struct audio_device *adev)  
{  
    // ...
```

```
adev->dev_out[SND_OUT_SOUND_CARD_HDMI_ARC].id = "HDMI_ARC";  
// ...  
}
```

验证

此时 Android 作为 TV, 通过 HDMI 线缆连接功放或回音壁音响, 功放或回音壁音响设置为 TV 或 Auto 模式

务必按照下面的顺序, 依次验证

确认 ARC 链路状态

通过 `dumpsys hdmi_control` 确认链路状态

```
# dumpsys hdmi_control  
// ...  
mMhlController:  
HDMI CEC Network  
  mPortInfo:  
    port_id: 1, type: HDMI_OUT, address: 0x0000, cec: true, arc: false, mhl:  
false  
  // ARC port 信息  
  port_id: 2, type: HDMI_IN, address: 0x3000, cec: true, arc: true, mhl: false  
HdmiCecLocalDevice #0:  
  // ...  
  // 已建立 ARC 物理链路  
  mArcEstablished: true  
  // ...  
mDeviceInfos:  
  CEC: logical_address: 0x00 device_type: 0 cec_version: 5 vendor_id: 1  
display_name: CTouchDisplay power_status: 0 physical_address: 0x0000 port_id: 1  
  // 连接 ARC port 的 soundbar 或功放的设备信息  
  CEC: logical_address: 0x05 device_type: 5 cec_version: 5 vendor_id: 5787878  
display_name: NS-R5101 power_status: -1 physical_address: 0x3000 port_id: 2  
mCecController:  
  CEC message history:  
    // ...  
    // 1. 连接 soundbar 或功放, ARC 建立的流程是在 HPD 为高之后发起  
    [H] time=2023-02-02 04:12:38 hotplug port=1 connected=true  
    // ...  
    // 2. 发现阶段: TV 获取已连接设备的 physical address, OSD name, vendor ID, Audio  
Mode Status  
    [S] time=2023-02-02 04:12:38 message=<Give System Audio Mode Status> 05:7D  
    [R] time=2023-02-02 04:12:40 message=<System Audio Mode Status> 50:7E:01  
    [R] time=2023-02-02 04:12:41 message=<Give Physical Address> 50:83  
    [S] time=2023-02-02 04:12:41 message=<Report Physical Address> 0F:84:00:00:00  
    [R] time=2023-02-02 04:12:41 message=<Report Physical Address> 5F:84:30:00:05  
    [S] time=2023-02-02 04:12:41 message=<Give Osd Name> 05:46  
    [R] time=2023-02-02 04:12:41 message=<Give Device Vendor Id> 50:8C  
    [S] time=2023-02-02 04:12:41 message=<Device Vendor Id> 0F:87:00:00:01  
    [R] time=2023-02-02 04:12:42 message=<Set Osd Name> 50:47 <Redacted len=8>  
    [S] time=2023-02-02 04:12:42 message=<Give Device Vendor Id> 05:8C  
    [R] time=2023-02-02 04:12:42 message=<Device Vendor Id> 5F:87:58:50:E6  
    [S] time=2023-02-02 04:12:42 message=<Give System Audio Mode Status> 05:7D
```

```
[S] time=2023-02-02 04:12:42 message=<Request ARC Initiation> 05:C3
[R] time=2023-02-02 04:12:42 message=<Device Vendor Id> 5F:87:58:50:E6
[R] time=2023-02-02 04:12:42 message=<Get Cec Version> 50:9F
[S] time=2023-02-02 04:12:42 message=<Cec Version> 05:9E:05
[R] time=2023-02-02 04:12:42 message=<System Audio Mode Status> 50:7E:01
// 3. 协商阶段: TV 接收到 soundbar 或功放发起 ARC 初始化请求
[R] time=2023-02-02 04:12:42 message=<Initiate ARC> 50:C0
// TV 回复 ARC 已初始化, 之后便建立了链路
[S] time=2023-02-02 04:12:42 message=<Report ARC Initiated> 05:C1
[R] time=2023-02-02 04:12:43 message=<Initiate ARC> 50:C0
[S] time=2023-02-02 04:12:43 message=<Report ARC Initiated> 05:C1
[R] time=2023-02-02 04:12:43 message=<Give Osd Name> 50:46
[S] time=2023-02-02 04:12:43 message=<Set Osd Name> 05:47 <Redacted len=13>
```

确认声卡可用性

通过 `tinyplay` 指定声卡播放一段 `wav` 格式的音频文件, 确认功放或回音壁音响是否接收到音频数据并输出

确认声卡序号

```
# cat /proc/asound/cards
0 [rockchiphdmiarc]: simple-card - rockchip,hdmiarc
rockchip,hdmiarc
...
```

`tinyplay` 指定声卡输出

```
# tinyplay xxx.wav -D 0
```

确认 Audio Policy 可用设备

ARC 链路建立了之后会在 Audio Policy 生成对应的 `devicePort`, 通过 `dumpsys media.audio_policy` 确认是否生成对应的可用设备

```
# dumpsys media.audio_policy
// ...
Available output devices (3):
1. Port ID: 2; "Speaker"; {AUDIO_DEVICE_OUT_SPEAKER, @:}
// ...
2. Port ID: 11; "HDMI Out"; {AUDIO_DEVICE_OUT_HDMI, @:}
// 此时已经生成 "HDMI ARC Out" 的 devicePort
3. Port ID: 3; "HDMI ARC Out"; {AUDIO_DEVICE_OUT_HDMI_ARC, @:}
```

确认 Audio Policy 路由状态

在 Android UI 点开任意 App, 保持播放任意音频文件, 通过 `dumpsys media.audio_policy` 确认是否路由到 ARC 的 `devicePort`

```
# dumpsys media.audio_policy
// ...
Audio Patches (1):
  1. owner uid 1041; handle 4; af handle 276
    [src 1] Mix Port ID: 1; I/O handle: 13;
    // 这里路由到了 ARC 的 devicePort
    [sink 1] Device Port ID: 3; {AUDIO_DEVICE_OUT_HDMI_ARC, @:}
```

Q&A

本节列举了适配过程中常见的问题, 可以根据问题现象或 logcat 报错的关键字段自检

CEC 相关问题

CEC 没有启用

```
# dumpsys hdmi_control
Can't find service: hdmi_control
```

请完成 CEC 功能的适配

没有 CEC 消息的收发

```
# dumpsys hdmi_control
// ...
HDMI CEC Network
// ...
CEC message history:
```

请完成 CEC 功能的适配

没有收到功放或回音壁音响的 CEC 消息

```
# dumpsys hdmi_control
// ...
HDMI CEC Network
// ...
CEC message history:
// ...
[S] time=2024-07-30 14:05:13 message=<Report Physical Address> 0F:84:FF:FF:00
()
[S] time=2024-07-30 14:05:13 message=<Device Vendor Id> 0F:87:00:00:01 ()
[S] time=2024-07-30 14:05:13 message=<Active Source> 0F:82:FF:FF ()
[S] time=2024-07-30 14:05:13 message=<Request Active Source> 0F:85 ()
[S] time=2024-07-30 14:13:18 message=<Routing Change> 0F:80:FF:FF:10:00 ()
[S] time=2024-07-30 14:13:19 message=<Set Stream Path> 0F:86:10:00 ()
```

- 确认设备的 ARC 口是否有接到功放或回音壁音响的 ARC 口
- 确认功放或回音壁音响是否有设置为 TV 或 Auto 模式

收到 <Initiate ARC> 后回复 <Feature Abort>

```
# dumpsys hdmi_control
// ...
HDMI CEC Network
mPortInfo:
    // 只有默认的 physical address 为 0.0.0.0 的 port
    port_id: 1, type: HDMI_OUT, address: 0x0000, cec: true, arc: true, mhl: false
CEC message history:
    // ...
    [R] time=2023-01-13 06:19:50 message=<Give Physical Address> 50:83
    [S] time=2023-01-13 06:19:50 message=<Report Physical Address> 0F:84:00:00:00
    [R] time=2023-01-13 06:19:50 message=<Report Physical Address> 5F:84:30:00:05
    // ...
    [R] time=2023-01-13 06:19:53 message=<Initiate ARC> 50:C0
    [S] time=2023-01-13 06:19:53 message=<Feature Abort>
// ...
```

设备上报的 physical address 为 3.0.0.0 (5F:84: 30:00 :05)

由于 framework 未配置和生成对应的 port 信息, 在 HDMI CEC HAL 中添加即可

功放或回音壁没有发起 <Initiate ARC>

```
# dumpsys hdmi_control
// ...
HDMI CEC Network
// ...
CEC message history:
    // ...
    [R] time=2023-01-13 06:22:54 message=<Give Physical Address> 50:83
    [S] time=2023-01-13 06:22:54 message=<Report Physical Address> 0F:84:00:00:00
    [R] time=2023-01-13 06:22:54 message=<Report Physical Address> 5F:84:00:00:05
    // ...
    [S] time=2023-01-13 06:22:55 message=<Request ARC Initiation> 05:C3
    // ...
    [R] time=2023-01-13 06:22:57 message=<Feature Abort> 50:00:C3:04
    // ...
```

设备上报的 physical address 为 0.0.0.0 (5F:84: 00:00 :05), 明显是 HDMI RX/Switch 分配的 physical address 不正确

修改分配给这个物理端口的 physical address, 必须为其分配直连的地址, 比如 x.0.0.0

HPD 频繁被 toggle, 声音无法保持输出

```
# dumphsys hdmi_control
// ...
HDMI CEC Network
// ...
CEC message history:
// ...
// 查看时间戳, HPD 有规律性地被 toggle
[H] time=2023-02-02 04:15:15 hotplug port=1 connected=false
[H] time=2023-02-02 04:15:16 hotplug port=1 connected=true
// ...
[H] time=2023-02-02 04:15:25 hotplug port=1 connected=false
[H] time=2023-02-02 04:15:26 hotplug port=1 connected=true
// ...
```

由于只有 ARC 链路, 在没有 TMDS 信号的情况下老版本的驱动会一直 toggle HPD

ARC 物理链路一旦建立, 需要保持 HPD 为高, 根据使用硬件设计或使用方案, 可以参考如下方法 workaround

- HDMI RX 驱动

```
diff --git a/drivers/media/platform/rockchip/hdmi/rk_hdmi.c
b/drivers/media/platform/rockchip/hdmi/rk_hdmi.c
index fd6bc391253bc..a1b346adcd6dc 100644
--- a/drivers/media/platform/rockchip/hdmi/rk_hdmi.c
+++ b/drivers/media/platform/rockchip/hdmi/rk_hdmi.c
@@ -32,6 +32,8 @@
#include <linux/seq_file.h>
#include <linux/v4l2-dv-timings.h>
#include <linux/workqueue.h>
#include <media/cec.h>
#include <media/cec-notifier.h>
#include <media/v4l2-common.h>
@@ -210,6 +212,7 @@ struct rk_hdmi_dev {
    struct regmap *grf;
    struct regmap *vol_grf;
    struct rk_hdmi_hdcp *hdcp;
    void __iomem *regs;
    int edid_version;
    int audio_present;
@@ -230,6 +233,8 @@ struct rk_hdmi_dev {
    bool cec_enable;
    bool hpd_on;
    bool force_off;
+   bool arc_enable;
+   bool earc_enable;
    u8 hdcp_enable;
    u32 num_clks;
    u32 edid_blocks_written;
@@ -2674,7 +2679,7 @@ static void hdmi_plugin(struct rk_hdmi_dev *hdmi_dev)
    hdmi_phy_config(hdmi_dev);
    hdmi_audio_setup(hdmi_dev);
    ret = hdmi_wait_lock_and_get_timing(hdmi_dev);
```

```

-   if (ret) {
+   if (ret && !hdmirx_dev->arc_enable) {
        hdmirx_plugin(hdmirx_dev);
        schedule_delayed_work_on(hdmirx_dev->bound_cpu,
                                &hdmirx_dev->delayed_work_hotplug,
@@ -3490,16 +3518,50 @@ static ssize_t status_store(struct device *dev,
        return count;
    }

+static ssize_t arc_enable_show(struct device *dev,
+                               struct device_attribute *attr, char *buf)
+{
+    struct rk_hdmirx_dev *hdmirx_dev = dev_get_drvdata(dev);
+
+    if (!hdmirx_dev)
+        return -EINVAL;
+
+    return snprintf(buf, PAGE_SIZE, "%d\n",
+                   hdmirx_dev->arc_enable ? 1 : 0);
+}
+
+static ssize_t arc_enable_store(struct device *dev,
+                               struct device_attribute *attr,
+                               const char *buf, size_t count)
+{
+    struct rk_hdmirx_dev *hdmirx_dev = dev_get_drvdata(dev);
+    bool enabled;
+    int ret;
+
+    if (!hdmirx_dev)
+        return -EINVAL;
+
+    ret = kstrtobool(buf, &enabled);
+    if (ret)
+        return ret;
+
+    hdmirx_dev->arc_enable = enabled;
+
+    return count;
+}
+
+static DEVICE_ATTR_RO(audio_rate);
+static DEVICE_ATTR_RO(audio_present);
+static DEVICE_ATTR_RW(edid);
+static DEVICE_ATTR_RW(status);
+static DEVICE_ATTR_RW(arc_enable);

static struct attribute *hdmirx_attrs[] = {
    &dev_attr_audio_rate.attr,
    &dev_attr_audio_present.attr,
    &dev_attr_edid.attr,
    &dev_attr_status.attr,
+    &dev_attr_arc_enable.attr,
    NULL
};

ATTRIBUTE_GROUPS(hdmirx);

```


- RK628 驱动

```
diff --git a/drivers/media/i2c/rk628/rk628_csi_v4l2.c
b/drivers/media/i2c/rk628/rk628_csi_v4l2.c
index 5a81de8e14c6..8d33f3b28ae2 100644
--- a/drivers/media/i2c/rk628/rk628_csi_v4l2.c
+++ b/drivers/media/i2c/rk628/rk628_csi_v4l2.c
@@ -3132,12 +3132,48 @@ static ssize_t audio_present_show(struct device *dev,
                                rk628_hdmirx_audio_present(csi->audio_info) : 0);
    }

+static ssize_t arc_enable_show(struct device *dev,
+                                struct device_attribute *attr, char *buf)
+{
+    struct rk628_csi *csi = dev_get_drvdata(dev);
+    struct rk_hdmirx_dev *hdmirx_dev = dev_get_drvdata(dev);
+
+    if (!hdmirx_dev)
+        return -EINVAL;
+
+    return snprintf(buf, PAGE_SIZE, "%d\n",
+                    rk628_hdmirx_get_arc_enable(csi->audio_info) ? 1 : 0);
+}
+
+static ssize_t arc_enable_store(struct device *dev,
+                                struct device_attribute *attr,
+                                const char *buf, size_t count)
+{
+    struct rk628_csi *csi = dev_get_drvdata(dev);
+    struct rk_hdmirx_dev *hdmirx_dev = dev_get_drvdata(dev);
+    bool enabled;
+    int ret;
+
+    if (!hdmirx_dev)
+        return -EINVAL;
+
+    ret = kstrtobool(buf, &enabled);
+    if (ret)
+        return ret;
+
+    rk628_hdmirx_set_arc_enable(csi->audio_info, enabled);
+
+    return count;
+}
+
static DEVICE_ATTR_RO(audio_rate);
static DEVICE_ATTR_RO(audio_present);
+static DEVICE_ATTR_RW(arc_enable);

static struct attribute *rk628_attrs[] = {
    &dev_attr_audio_rate.attr,
    &dev_attr_audio_present.attr,
+    &dev_attr_arc_enable.attr,
    NULL
}
```

```

};
ATTRIBUTE_GROUPS(rk628);
diff --git a/drivers/media/i2c/rk628/rk628_hdmirx.c
b/drivers/media/i2c/rk628/rk628_hdmirx.c
index ef69600a5340..72a0ddada9d2 100644
--- a/drivers/media/i2c/rk628/rk628_hdmirx.c
+++ b/drivers/media/i2c/rk628/rk628_hdmirx.c
@@ -42,6 +42,7 @@ struct rk628_audioinfo {
    bool fifo_ints_en;
    bool ctsn_ints_en;
    bool audio_present;
+   bool arc_en;
    struct device *dev;
};

@@ -639,6 +640,28 @@ int rk628_hdmirx_audio_fs(HAUDINFO info)
}
EXPORT_SYMBOL(rk628_hdmirx_audio_fs);

+bool rk628_hdmirx_get_arc_enable(HAUDINFO info)
+{
+   struct rk628_audioinfo *aif = (struct rk628_audioinfo *)info;
+
+   if (!aif)
+       return false;
+
+   return aif->arc_en;
+}
+EXPORT_SYMBOL(rk628_hdmirx_get_arc_enable);
+
+int rk628_hdmirx_set_arc_enable(HAUDINFO info, bool enabled)
+{
+   struct rk628_audioinfo *aif = (struct rk628_audioinfo *)info;
+
+   if (!aif)
+       return false;
+
+   return aif->arc_en = enabled;
+}
+EXPORT_SYMBOL(rk628_hdmirx_set_arc_enable);
+
void rk628_hdmirx_audio_i2s_ctrl(HAUDINFO info, bool enable)
{
    struct rk628_audioinfo *aif = (struct rk628_audioinfo *)info;
diff --git a/drivers/media/i2c/rk628/rk628_hdmirx.h
b/drivers/media/i2c/rk628/rk628_hdmirx.h
index 1f0a302a75a8..15538e8ab6bf 100644
--- a/drivers/media/i2c/rk628/rk628_hdmirx.h
+++ b/drivers/media/i2c/rk628/rk628_hdmirx.h
@@ -504,6 +504,8 @@ void rk628_hdmirx_audio_cancel_work_rate_change(HAUDINFO
info, bool sync);
bool rk628_hdmirx_audio_present(HAUDINFO info);
int rk628_hdmirx_audio_fs(HAUDINFO info);
void rk628_hdmirx_audio_i2s_ctrl(HAUDINFO info, bool enable);
+bool rk628_hdmirx_get_arc_enable(HAUDINFO info);
+int rk628_hdmirx_set_arc_enable(HAUDINFO info, bool enabled);

```

```
/* for audio isr process */
bool rk628_audio_fifoents_enabled(HAUDINFO info);
```

```
diff --git a/drivers/media/i2c/rk628/rk628_csi_v4l2.c
b/drivers/media/i2c/rk628/rk628_csi_v4l2.c
index 8d33f3b28ae2..9555ad2163b5 100644
--- a/drivers/media/i2c/rk628/rk628_csi_v4l2.c
+++ b/drivers/media/i2c/rk628/rk628_csi_v4l2.c
@@ -494,6 +494,9 @@ static void rk628_hdmirx_config_all(struct v4l2_subdev *sd)
    }

    if (ret < 0 || rk628_hdmirx_scdc_ced_err(csi->rk628)) {
+       if (rk628_hdmirx_get_arc_enable(csi->audio_info))
+           return;
+
        rk628_hdmirx_pluginout(sd);
        csi->lock_fail_time++;
        v4l2_dbg(1, debug, sd, "%s: lock fail time: %d\n",
```

- 设置权限

```
device/rockchip/common/rootdir/init.rk30board.rc
```

```
# for HDMI RX
chown system system /sys/class/hdmirx/hdmirx/arc_enable
chmod 0660 /sys/class/hdmirx/hdmirx/arc_enable

# for RK628
chown system system /sys/class/hdmirx/rk628/arc_enable
chmod 0660 /sys/class/hdmirx/rk628/arc_enable
```

- HDMI CEC HAL 设置对应的属性

```
static void hdmi_cec_set_audio_return_channel(const struct hdmi_cec_device* dev,
int port_id, int flag)
{
    // ...
    FILE* file = NULL;

    // for HDMI RX
    file = fopen("/sys/class/hdmirx/hdmirx/arc_enable", "w");
    if (file) {
        fprintf(file, "%d", flag);
        fclose(file);
    }

    // for RK628
    file = fopen("/sys/class/hdmirx/rk628/arc_enable", "w");
    if (file) {
        fprintf(file, "%d", flag);
        fclose(file);
    }
}
```

```
// ...  
}
```

Audio 相关问题

无法生成 ARC 的声卡

- 检查项目使用的 dts, 确认对应的硬件接口的节点是否有启用 status = "okay"
- 检查项目使用的 dts, 确认引脚复用 pinctrl-0 是否和其他节点冲突

通过 tinyply 指定 ARC 的声卡播放音频, 功放或回音壁音响没有输出

- 确认是否已经建立 ARC 物理链路, 参考 [3.1 小节](#)
- 检查项目使用的 dts, 确认引脚复用 pinctrl-0 是否和实际硬件一致